```python
# Skeleton Program for the AQA AS Summer 2026 examination
# this code should be used in conjunction with the Preliminary Material
# written by the AQA Programmer Team
# developed using PyCharm Community Edition 2022

# Version number 0.0.1

import random

TILE = "[X]"
NO_TILE = "[ ]"

Width = 4
Height = 4
Board = []

def ResetBoard(RandomOption):
    global Board
    Board = []
    for Row in range(Height):
        Board.append([])
        for Column in range(Width):
            if RandomOption:
                Board[Row].append(GetRandomTile())
            else:
                Board[Row].append(TILE)

def GetRandomTile():
    Rand = random.randint(1, 10)
    if Rand < 10:
        return TILE
    else:
        return NO_TILE

def DisplayState(PlayerNumber):
    print("-------------------------")
    print(f"Player {PlayerNumber}'s turn")
    print()
    DisplayBoard()

def DisplayBoard():
    print("  ", end='')
    for Column in range(Width):
        Header = chr(Column + 65)
        print(f"  {Header} ", end='')
    print()
    for Row in range(Height):
        print(Row + 1, end=" ")
        for Column in range(Width):
            print(f" {Board[Row][Column]}", end='')
        print()

def ConvertRefToCoords(Ref):
    Column = ord(Ref[0]) - 65
    Row = int(Ref[1]) - 1
    if Row < Height and Column < Width:
        Coords = [Row, Column]
    else:
        Coords = []
    return Coords

def ConvertCoordsToRef(Row, Column):
    Ref = ""
    if Row < Height and Column < Width:
        Letter = chr(Column + 65)
        Number = Row + 1
        Ref = Letter + str(Number)
    return Ref
```

```python

def ProcessMove(Move):
    if "-" in Move:
        DashPos = Move.index("-")
        FirstRef = Move[0:DashPos]
        SecondRef = Move[DashPos + 1:]
    else:
        FirstRef = Move
        SecondRef = Move
    StartCoords = ConvertRefToCoords(FirstRef)
    EndCoords = ConvertRefToCoords(SecondRef)
    if len(StartCoords) == 0 or len(EndCoords) == 0:
        return False
    if StartCoords[0] != EndCoords[0] and StartCoords[1] != EndCoords[1]:
        return False
    if StartCoords[0] == EndCoords[0]:
        ToRemove = EndCoords[1] - StartCoords[1] + 1
        for Cell in range(StartCoords[1], EndCoords[1] + 1):
            if Board[StartCoords[0]][Cell] == TILE:
                ToRemove -= 1
        if ToRemove == 0:
            for Cell in range(StartCoords[1], EndCoords[1] + 1):
                Board[StartCoords[0]][Cell] = NO_TILE
        else:
            return False
    else:
        ToRemove = EndCoords[0] - StartCoords[0] + 1
        for Cell in range(StartCoords[0], EndCoords[0] + 1):
            if Board[Cell][StartCoords[1]] == TILE:
                ToRemove -= 1
        if ToRemove == 0:
            for Cell in range(StartCoords[0], EndCoords[0] + 1):
                Board[Cell][StartCoords[1]] = NO_TILE
        else:
            return False
    return True

def SetBoardSize():
    global Width
    global Height
    Width = int(input("Specify board width: "))
    Height = int(input("Specify board height: "))

def DisplayMenu(RandomOption):
    print("1 - Start game")
    print(f"2 - Set board size (currently {Width} x {Height})")
    print(f"3 - Toggle random option (currently {RandomOption})")
    print("4 - Load test board (4 x 4)")
    print("9 - Quit")

def LoadTestBoard():
    global Width
    global Height
    Width = 4
    Height = 4
    ResetBoard(False)
    ProcessMove("A1-A4")
    ProcessMove("B1-B4")
    ProcessMove("C1-C4")
    ProcessMove("D1-D2")
```

```python
def CheckGameOver():
    Remaining = 0
    for Row in range(Height):
        for Column in range(Width):
            if Board[Row][Column] == TILE:
                Remaining += 1
    if Remaining == 1:
        return True
    else:
        return False

def PlayGame():
    print(f"Valid moves are within the range A1-{ConvertCoordsToRef(Height - 1, Width - 1)}")
    GameOver = False
    NextPlayer = 1
    while not GameOver:
        DisplayState(NextPlayer)
        print()
        IsValid = False
        while not IsValid:
            Move = input("Enter move: ")
            IsValid = ProcessMove(Move)
            if not IsValid:
                print("Not a valid move - try again")
        NextPlayer = NextPlayer % 2 + 1
        if CheckGameOver():
            GameOver = True
            print(f"Game over - player {NextPlayer % 2 + 1} wins")
            print()
            DisplayBoard()
            print()
            print("Press enter to continue")
            input()
```

```python
def Main():
    Playing = True
    RandomOption = False
    while Playing:
        ExitMenu = False
        while not ExitMenu:
            print()
            DisplayMenu(RandomOption)
            UserInput = int(input("Enter a choice: "))
            if UserInput == 1:
                ExitMenu = True
                ResetBoard(RandomOption)
            elif UserInput == 2:
                SetBoardSize()
            elif UserInput == 3:
                RandomOption = not RandomOption
            elif UserInput == 4:
                ExitMenu = True
                LoadTestBoard()
            elif UserInput == 9:
                print("Thank you for playing")
                ExitMenu = True
                Playing = False
        if Playing:
            PlayGame()
    print("Press enter to continue")
    input()

if __name__ == "__main__":
    Main()
```